

Learning Lab : La Recursivite

Maitriser l'inconnu en developpement Full Stack

Sommaire

1. **Icebreaker** : C'est quoi pour vous ?
2. **Definition** : Le concept d'auto-appel.
3. **Duel** : Boucles vs Récursivité.
4. **Anatomie** : La Stack et le Cas de Base.
5. **Dangers** : Stack Overflow.
6. **Expertise** : Le Backtracking.
7. **Workshop** : Mise en pratique.

Question

Selon vous, c'est quoi la recursivite ?

(Repondez par un mot, une image ou une situation du quotidien)

1. Qu'est-ce que la recursivite ?

La recursivite est une methode de resolution de problemes ou une fonction **s'appelle elle-meme** pour resoudre des sous-problemes identiques au probleme initial.

"Pour comprendre la recursivite, il faut d'abord comprendre la recursivite."

2. Comparaison : Boucles vs Récursivité

Critère	Boucle (For/While)	Récursivité
Approche	Iterative (pas à pas)	Declarative (diviser pour régner)
Données	Listes plates, Tableaux	Arbres, JSON imbriquée, DOM
Complexité	Horizontale (imbrications)	Verticale (profondeur)
Usage	Index connu	Profondeur inconnue

3. Exemple : Somme recursive

Plutot qu'une boucle `for`, on decompose le probleme.

```
function calculerSomme(arr) {  
  // Cas de base : le tableau est vide  
  if (arr.length === 0) return 0;  
  
  // Appel recursif : premier element + somme du reste  
  return arr[0] + calculerSomme(arr.slice(1));  
}
```

4. La Stack (Pile d'execution)

L'ordinateur utilise une Stack (LIFO : Last In, First Out) pour gerer les appels.

1. Chaque appel "empile" un nouveau contexte en memoire.
2. L'appel precedent est mis en pause tant que le suivant n'est pas termine.
3. Une fois le cas de base atteint, la pile se "vide" (depilement) pour retourner le resultat final.

5. Les deux piliers de la fonction

Pour qu'une fonction recursive soit valide, elle doit respecter deux regles :

1. **Le Cas de Base** : la condition de sortie qui arrete l'auto-appel.
2. **L'Appel Recursif** : l'appel a soi-meme qui doit nous rapprocher du cas de base.

6. Le danger : Stack Overflow

Si la pile devient trop grande (trop d'appels sans condition de sortie), le navigateur sature la memoire.

```
function erreur() {  
  return erreur();  
}  
  
// RangeError: Maximum call stack size exceeded
```

C'est l'equivalent d'une boucle infinie, mais qui sature la memoire vive.

7. Le Backtracking

C'est la puissance de la recursivite : explorer une branche, et si elle echoue (cul-de-sac), revenir en arriere pour explorer la branche suivante.

Usages typiques :

1. Pathfinding (Snake, labyrinthes)
2. Resolution de Sudoku
3. Recherche dans des arborescences complexes

Workshop : Mise en pratique

Objectif

Développer une bibliothèque d'outils capables d'analyser et d'afficher une structure de données dont la profondeur est imprévisible.

Exercices (En continuité)

1. **L'Init** : Créez une fonction `main()` asynchrone pour charger `data.json`.
2. **Le Décompte** : Créez une fonction récursive `lancement(n)` qui affiche un compte à rebours en console avant de traiter les données.
3. **L'Analyseur** : Créez `compterPages(data)` pour retourner le nombre total de nœuds dans l'arbre.
4. **Le Chercheur** : Créez `trouverUrl(data, nom)` qui utilise le backtracking pour